

1. Титульная информация.

ФИО: Юсупов Арсений Ринатович

Группа: СКБ251

Дисциплина: Языки программирования

Тема работы: Многопоточность в C++

2. Постановка задачи.

Необходимо реализовать консольное приложение на C++ (C++11 и выше), моделирующее обработку задач несколькими потоками.

3. Описание реализации.

- Главный поток генерирует набор исходных данных и добавляет их в потокобезопасную очередь. Далее создаются независимые потоки, работающие параллельно. В программе реализован класс `TaskQueue`. В нем написаны методы добавления и извлечения задач, синхронизации данных. Также в программе реализована функция `void worker(unsigned int id, TaskQueue& q)`. Она имитирует обработку задачи и выполняет потокобезопасный вывод в консоль. В `main()` определяется количество потоков процессора при помощи `hardware_concurrency()`, но если оно вернуло 0, то программа определяет количество потоков процессора как 1. Для Windows вывод в консоли настроен в кодировке UTF-8.
- Потокобезопасная очередь реализована с помощью `std::mutex`. `std::queue` не обладает потокобезопасностью, что приводит к гонке данных, если происходит одновременный доступ нескольких потоков без синхронизации данных. В методе `push(const int task)` класса `TaskQueue` поток захватывает `mutex` через `std::lock_guard`, добавляет задачу и пробуждает один из ожидающих потоков при помощи `cv.notify_one()`. В методе

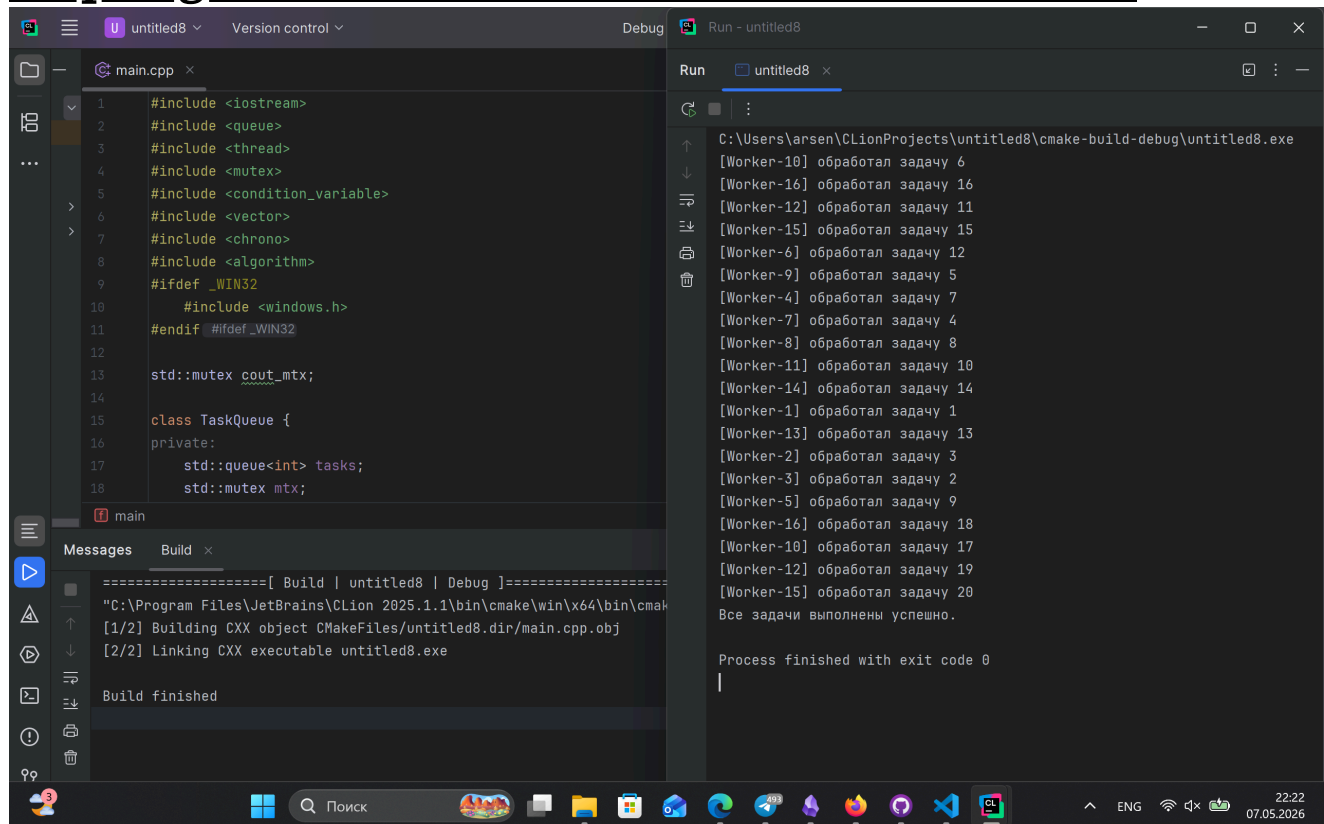
`pop(int& task)` также поток захватывает `mutex` через `std::unique_lock`. `std::condition_variable` требует `unique_lock`, так как нам необходимо атомарно освободить `mutex` и перевести поток в состоянии ожидания при вызове `wait()`. В методе `wait()` мы передаем предикат в лямбде-выражении, что защитит нас от ложных пробуждений и сработает только в случае, когда у нас есть задачи или установлен флаг `stop`, а дальше идет проверка очереди, которая позволяет обработать корректно. В методе `stopQueue()` главный поток захватывает мьютекс, устанавливает флаг `stop = true` и вызывает `cv.notify_all()`, пробуждая все рабочие потоки для корректного завершения. Мьютекс здесь необходим чтобы изменить общее состояние `stop`. Предикат в методе `pop(int& task)` в `wait()` проверяет условие `!tasks.empty() || stop`, которое не дает потоку обратиться к пустой очереди и зависнуть.

- `std::condition_variable` применяется, чтобы не использовать `busy-wait`, которое запрещено по условиям. `std::condition_variable` помогает нам не загружать потоки, которые не задействуются при выполнении программы. Применяется она в методе `push(const int task) notify_one()`, чтобы пробудить один из спящих потоков, в методе `stopQueue()` `notify_all` пробуждает все ожидающие потоки. В методе `pop(int& task)` мы используем `wait()` и предикат в виде лямбда-выражения, чтобы не допустить ложных срабатываний и не обращаться к пустой очереди.
- Поток завершается в функции `worker()`. В классе `TaskQueue` в методе `stopQueue()` поток захватывает мьютекс и устанавливает флаг `stop = true`. Через `std::condition_variable` уведомляются все потоки, чтобы завершить спящие потоки. Далее потоки продолжают работу в методе `pop(int& task)`, который возвращает `true`, если есть задачи, а если нет задач и установлен флаг `stop = true`, то возвращает `false`. Как только метод вернет `false`, тогда это приводит к выходу из цикла

worker. После выхода рабочих потоков из цикла главный поток корректно завершает их с помощью `join()`.

4. Ссылка на github:

<https://github.com/KSASaerocoolVX/homework>



```
#include <iostream>
#include <queue>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <vector>
#include <chrono>
#include <algorithm>
#ifdef _WIN32
#include <windows.h>
#endif
std::mutex cout_mtx;

class TaskQueue {
private:
    std::queue<int> tasks;
    std::mutex mtx;
};

int main() {
    TaskQueue tq;
    tq.tasks.push(1);
    tq.tasks.push(2);
    tq.tasks.push(3);
    tq.tasks.push(4);
    tq.tasks.push(5);
    tq.tasks.push(6);
    tq.tasks.push(7);
    tq.tasks.push(8);
    tq.tasks.push(9);
    tq.tasks.push(10);
    tq.tasks.push(11);
    tq.tasks.push(12);
    tq.tasks.push(13);
    tq.tasks.push(14);
    tq.tasks.push(15);
    tq.tasks.push(16);
    std::vector<std::thread> workers;
    for (int i = 0; i < 16; ++i) {
        workers.emplace_back([i, &tq]() {
            while (true) {
                int task;
                {
                    std::lock_guard<std::mutex> lock(tq.mtx);
                    if (tq.tasks.empty()) {
                        break;
                    }
                    task = tq.tasks.front();
                    tq.tasks.pop();
                }
                std::cout << "[Worker-" << i << "] обработал задачу " << task << "\n";
            }
        });
    }
    for (auto& w : workers) {
        w.join();
    }
    std::cout << "Все задачи выполнены успешно.\n";
    return 0;
}
```

Build finished

Process finished with exit code 0

5. Выводы:

- Я изучил основы многопоточности, синхронизацию потоков, разобрался в использовании библиотек: `std::mutex`, `std::thread`, `std::condition_variable`. Узнал как правильно решать проблему busy-wait при ожидании новых задач. Как совмещать использование `mutex` и `std::condition_variable` для эффективного использования ресурсов процессора. В программе рабочие потоки обрабатывают задачи параллельно, а главный поток корректно ожидает их завершения с помощью `join()`